

Cascading Failure Simulator

"SimCity for Infrastructure Resilience"

1. The Problem, In Plain Words

Cities are held together by systems that depend on each other — power lines, roads, hospitals, water pipes, bridges. Right now, most cities watch these systems one at a time. Someone checks "is this power station okay?" Someone else checks "is this road okay?" But nobody easily sees the connections between them.

Here's why that's dangerous: when one thing breaks, it doesn't stay broken alone. A power station going down can knock out a hospital's main power. The hospital switches to backup generators. If the generators run out of fuel before power is restored, patients are at risk — but nobody saw that coming because nobody was watching the *chain*, only the individual pieces.

This is called a **cascading failure** — one failure causing another, causing another, like dominoes.

Our goal: build a tool that lets city planners see this domino effect *before* it happens in real life, test out different failures, and figure out where to spend limited money to prevent the worst outcomes.

2. The Big Idea

We are building a simulation — think of it like **The Sims, but for a city's infrastructure** instead of people's lives.

Instead of managing a Sim's hunger and happiness, you're managing a city's power grid, roads, hospitals, and water systems. You can:

- Look at how everything connects

- Break something on purpose (simulate a failure) and watch what happens

- Spend a budget to fix or protect things

- Get advice from an AI on what the *smartest* fix would have been

- Compare different plans side by side

It looks and feels like a city-building game, but underneath, it's running a real simulation based on real infrastructure logic.

3. What Makes This Different From a Normal "Infrastructure Map" Tool

Most tools that try to solve this problem show you a **diagram** — circles (representing buildings, stations, etc.) connected by lines, with some circles turning red when they "fail." It looks like a school science project: technically correct, but nobody wants to interact with it.

We are instead building something that:

Looks like a real, playable city (isometric, low-poly, game-like — not a diagram)

Shows failures happening over time, not instantly

Tells you how many people are affected, not just how many buildings

Lets you spend a budget to fix things, like a city-building game

Has an AI advisor that grades your decisions against the mathematically best possible plan

Shows honest uncertainty instead of pretending its numbers are exact

Each of these is explained below.

4. Feature-by-Feature Explanation

4.1 Explore the Network

Every building or piece of infrastructure (hospital, power station, road junction, water pump) can be clicked on. When you click it, you see:

What it depends on (e.g., a hospital depends on a specific power line)

What depends on it (e.g., a power station might supply 3 hospitals and 500 homes)

Its current status: working fine, running on backup, or failed

Live stats: current load, capacity, and criticality score

Why it matters: You can't fix a system you don't understand. This lets planners actually see the hidden dependencies that normally only exist in someone's head or in scattered documents.

4.2 Trigger a Failure

You click on one or more things and "break" them — simulate a power station going offline, a bridge collapsing, a water main bursting.

Why it matters: This lets planners run "what if" tests safely. What if this substation fails during a heatwave? What if two bridges are out at once during a flood? You can test disaster scenarios without an actual disaster happening.

4.3 Time-Based Failure (Not Instant On/Off) — Key Differentiator #1

Most tools treat failure as instant: something is either "working" or "broken," like a light switch. That's not how real infrastructure works.

In real life, when a hospital loses main power, it doesn't shut down immediately — it switches to a backup generator. That generator has a fuel tank that lasts a certain number of hours. If power isn't restored before the fuel runs out, *then* the hospital actually fails.

So in our simulation:

Things don't fail instantly — they **degrade** over time (working → running on backup → critically low → failed)

There's a **timeline scrubber** you can play, pause, step through, or speed up (1x/2x/3x, exactly like The Sims), watching the situation get worse (or get fixed) minute by minute or hour by hour

This tells planners something much more useful than "this will fail" — it tells them **how much time they have to respond** before it does

Why it matters: A planner's real question isn't "will this fail?" — it's "how long do I have to send help before it fails?" Our simulation answers that question directly. Almost every competing team will build simple binary up/down propagation; very few will bother modeling time, which makes this a high-value, low-competition differentiator.

4.4 Human Impact, Not Just Building Impact — Key Differentiator #2

Most tools would report: "12 infrastructure assets failed."

We instead report: **how many people were actually affected, and how badly**, broken down by vulnerability class. For example:

How many patients were on life support in a hospital that lost power

How many households lost clean water

How much longer an ambulance would take to reach someone because a road was blocked

Why it matters: Planners don't manage buildings — they manage the wellbeing of people living in the city. A number like "40,000 people without water for 6 hours" means something. "12 assets failed" does not. This also ties directly to the "Sustainable Cities and Communities" framing of the problem — most teams will treat this as a generic graph problem and miss the community/SDG angle entirely. Judges scoring against that lens will notice who actually engaged with "communities," not just "infrastructure."

4.5 Finding the Most Important Assets

The system automatically calculates which pieces of infrastructure are the most critical — meaning, if *this* one thing failed, it would cause the most damage overall.

Instead of just counting how many connections something has (which is the simple, common approach — raw graph centrality), we actually **simulate removing it** and measure the real damage that would follow.

This gives a much more honest answer about what's actually important versus what just looks important on paper.

Why it matters: Cities don't have unlimited money. This tells planners exactly where a single point of failure could cause a disaster, so they know where protecting things first will matter most.

4.6 Build Mode: Spend a Budget to Fix Things — Key Differentiator #3 (the centerpiece)

This is the part that makes it feel like a real game, like SimCity.

You're given a budget, shown as a counter in the corner (like Simoleons). You spend it from a palette of purchasable interventions, each with a cost and a stated effect:

A backup generator for a hospital (extends time-to-failure)

A second power line as a backup route (adds redundancy)

A stronger, reinforced bridge (removes a failure mode)

An alternate road route (cuts emergency response time)

A water storage tank (extends time-to-failure for water-dependent buildings)

You drag these directly onto the map, the same way you'd place a building in a city-building game. The underlying graph model updates live (new nodes/edges with new capacity attributes), and it actually changes how the simulation behaves — if you protected a hospital with a backup generator, it won't fail as fast next time the power goes out.

Why it matters: This turns the tool from something you *read* into something you *do*. Planners can directly test: "If I only have \$2 million to spend this year, what should I protect first?" — and immediately see the results of their choice play out.

4.7 Intervention ROI + AI Advisor: How Good Was Your Plan?

This is where the recommendation engine does real work, not just decoration.

Rather than only ranking assets by importance, the system runs the removal-impact simulation **in reverse**, as an investment allocator: "given this budget, which combination of interventions minimizes worst-case cascade damage?" After you place your own budget by hand, an AI advisor compares your plan against that mathematically optimal placement for the same money — and highlights the alternate plan directly on the map so you can see exactly what you would have done differently.

For example, it might say:

"Your plan reduced the worst-case damage by 34%. The best possible plan for this budget would have reduced it by 61% — here's what it would have looked like instead."

Why it matters: This directly answers the core question every planner actually has — "where would spending money make the biggest difference?" — which is literally the problem statement's own stated

goal, turned into a working feature instead of a sentence in a write-up. It also makes the experience feel like a game you're trying to beat, not a report you're reading.

4.8 Compare Different Plans Side by Side

You can save different scenarios — different failures, different budgets, different fixes — and compare them directly against each other. For example: "Plan A: fix the bridge first" vs. "Plan B: fix the power line first" — which one actually protects more people?

Why it matters: Real decisions are rarely about just one plan. Planners need to weigh options against each other, not just see one result in isolation.

4.9 Uncertainty, Not False Precision — Key Differentiator #4

Real infrastructure data is never perfectly known — capacity, load, and fuel/reserve levels are always estimates. Instead of pretending our numbers are exact, we model them as **ranges with confidence**, and run a Monte Carlo loop over the existing propagation rules to produce a probability spread of outcomes rather than one deterministic answer. For example, instead of "this cascades to 3 more nodes," the tool shows: "60% chance this cascades to 3 more nodes, 15% chance it reaches the hospital cluster."

Why it matters: This is more honest, and it reads as more technically sophisticated to judges, while being cheap to build — it's the same propagation engine, just sampled repeatedly over input ranges instead of run once on point estimates.

5. What It Actually Looks Like

Picture a small city viewed from an isometric angle (like old city-building games) — roads, a hospital building, power stations, water towers, all recognizable, in a clean low-poly art style.

A budget counter sits in a corner, like money in a city-building game

You can click any building to see its stats in a side panel (load %, dependents, criticality score, status, and — if failed — estimated population impact)

Playback controls (pause, play, speed up) let you watch a failure unfold over time

A build menu lets you drag new protections onto the map

Color changes show status: green (fine), yellow (degraded/on backup), red (failed)

After running a scenario, a summary panel shows the human impact numbers, the uncertainty spread, and the AI advisor's comparison, styled like a Sims advisor popup rather than a data table

It's meant to feel less like a spreadsheet and more like something you'd actually want to play with — which is exactly the point, because that's what makes people (including judges) actually engage with it instead of just skimming a report.

6. Technical Approach & Build Scope

The graph/simulation model (nodes, edges, capacity, degradation curves, intervention types) is the same underlying data structure regardless of how it's rendered — only the *presentation layer* needs to look like a game.

To hit this look realistically in hackathon time:

Engine: Unity, rendering the city ward as an isometric scene rather than a node-link diagram

Art: low-poly / flat-shaded assets (free low-poly city asset packs) rather than anything hand-modeled — enough for roads, substations, hospitals, and water towers to read clearly as real objects

Camera: a fixed isometric camera rather than a free-roaming 3D camera — much cheaper to make look good, and closer to actual Sims/city-builder framing

Engineering priority: spend the majority of build time on the simulation and recommendation engine being real and correct, not on graphics polish — judges reward working depth over visual fidelity once the "looks nice enough" bar is cleared

This keeps the ambitious visual pitch achievable without turning the project into an art-asset production problem.

7. Why This Approach Wins

Almost every team working on this same problem will likely build:

A static graph of circles and lines

A button that makes some circles turn red via simple threshold propagation

A static list titled "Top 5 Most Important Assets" ranked by degree centrality

That satisfies the requirements on paper, but it's forgettable, and it treats infrastructure failure as an instant, simple, perfectly-known thing — which it isn't in real life.

Our approach is different because:

It **looks and feels like something people want to interact with**, not read

It treats failure as something that **unfolds over time**, which is how it actually works in real cities

It measures success in **human terms** (people affected), not just technical terms (assets failed) — which directly matches the "Sustainable Cities and Communities" spirit of the problem

It turns the recommendation engine into an **active, game-like comparison** ("here's how close you got to the best plan") instead of a static report — built on a genuine reverse-simulation investment allocator, not just a criticality ranking

It's **honest about uncertainty** instead of presenting fabricated precision, which reads as more technically credible

It directly implements the problem statement's own stated goal — helping planners see "where intervention could have the greatest effect" — as a real, working feature, not just a sentence in our project write-up

8. One-Sentence Summary

We're building a Sims-style playable simulation of a city's infrastructure, where you can break things, watch the damage spread realistically over time with honest uncertainty, spend a budget to fix the most important weak points, and get AI advice on whether you spent your money in the smartest possible place.